CSE 3243 Web Programming Lab

Mini Project Report on

Quizy+

Intelligent Online Quiz and Examination System

SUBMITTED BY

Abhinav Mohapatra	Roll No: 41	Registration No: 230905372
Ayan Singh	Roll No: 42	Registration No: 230905390
Harsh Bhardwaj	Roll No: 56	Registration No: 230905454

Section: CSE-D

Under the Guidance of:

Ms. Athira Gopalakrishnan and Mrs. Sahana Roshan

School of Computer Engineering

Manipal Institute of Technology, Manipal, Karnataka - 576104

2025-26

Acknowledgement

We would like to express our sincere gratitude to the School of Computer Engineering, Manipal Institute of Technology, for providing the academic environment, laboratory infrastructure, and technical guidance required to complete this project.

We are deeply thankful to our faculty mentors, [Faculty Name 1] and [Faculty Name 2], for their continuous support, critical feedback, and encouragement throughout the design, implementation, testing, and documentation phases. Their guidance significantly improved the technical quality and research depth of this work.

We also acknowledge our peers and classmates for constructive discussions on software architecture, testing strategy, and user experience design. Their insights helped refine key features such as adaptive quiz progression, anti-cheating telemetry, and analytics reporting.

Finally, we thank our families for their constant encouragement and motivation throughout the project lifecycle.

Abstract

Quizy+ is a Django-based intelligent online quiz and examination platform designed to modernize digital assessment through secure exam delivery, adaptive question sequencing, automated scoring, and analytics-driven feedback. The system supports two primary roles - Admin and Student - with dedicated workflows for quiz authoring, participation, evaluation, and performance monitoring.

The platform is implemented using Python (Django MVT), HTML, CSS, and JavaScript, with SQLite as the default persistence layer and MySQL readiness for scaled deployment. Beyond baseline examination features, Quizy+ introduces adaptive difficulty progression, gamification mechanisms (points, badges, streaks, leaderboard), anti-cheating controls (tab-switch and fullscreen exit telemetry), and a live quiz mode for host-led sessions.

Comprehensive testing was conducted across unit, integration, edge-case, and security scenarios. The final system demonstrates strong modularity, maintainability, and production readiness, making it suitable for academic institutions, training organizations, and online learning programs.

Table of Contents

1. Introduction

2. Related Work

3. Problem Statement

4. Proposed Work

5. System Design

6. Implementation

7. Testing

8. Screenshots and Output

9. Conclusion and Future Enhancements

10. References

11. Web Development Project Journal Paper: Comprehensive Research and Implementation Report

1. Introduction

1.1 Background of Online Examination Systems

Educational institutions and professional training platforms increasingly rely on digital assessment systems for scalable, objective, and accessible evaluation. Traditional pen-and-paper examinations involve substantial overhead in question preparation, invigilation, manual grading, and result publication. These workflows are time-consuming, error-prone, and difficult to scale for large cohorts.

Online examination platforms provide centralized assessment management, instant evaluation support for objective questions, and data-rich performance insights. However, many available tools are either generic form builders or enterprise systems with limited customizability for intelligent assessment logic.

1.2 Motivation for Digital Assessment

The motivation behind Quizy+ is to build a practical, extensible, and academically relevant web platform that combines:

- secure exam conduction,
- adaptive learning-oriented assessment,
- meaningful analytics,
- and improved user engagement.

The project specifically addresses the need for a role-aware system where administrators can design high-quality quizzes and students can receive immediate, actionable feedback.

1.3 Problems with Traditional and Basic Digital Exams

Traditional and basic digital exam systems commonly suffer from the following limitations:

- delayed result generation,
- low personalization (static question difficulty),
- weak anti-cheating controls,
- fragmented student performance tracking,
- and low motivation for repeated participation.

These limitations reduce both operational efficiency and educational effectiveness.

1.4 Project Significance

Quizy+ contributes to the digital assessment domain by integrating foundational examination workflows with intelligence and engagement features in a single web application. Its significance lies in:

- automation of evaluation and reporting,
- adaptive difficulty for improved assessment granularity,
- secure and controlled attempt lifecycle,
- transparent analytics for both instructors and learners,
- and a modular architecture suitable for future research and deployment.

2. Related Work / Literature Review

2.1 Existing Platforms

Google Forms

Google Forms is widely used for quick quizzes and surveys. It supports objective question grading but lacks robust examination controls such as adaptive sequencing, role-based advanced analytics, and anti-cheating telemetry.

Moodle

Moodle is a mature LMS with quiz modules, question banks, and course integration. While feature-rich, it is often heavy for lightweight deployments and can require complex configuration for custom adaptive behavior and modern interaction patterns.

Kahoot

Kahoot provides strong live participation and gamified engagement but is optimized for synchronous sessions rather than comprehensive examination lifecycle management, adaptive scoring logic, and structured academic analytics.

Other LMS and Quiz Tools

Many platforms provide either strong content management or strong interaction design, but few provide a balanced combination of exam rigor, adaptive intelligence, and operational simplicity in a customizable codebase.

2.2 Identified Gaps

From the literature and platform comparison, key gaps include:

- limited adaptive question progression tied to user performance,
- inconsistent support for anti-cheating signals at attempt level,
- insufficient topic-wise and time-wise analytics for pedagogy,
- low extensibility for institution-specific policy controls,
- and weak integration between gamification and exam outcomes.

2.3 Gap Addressed by Quizy+

Quizy+ addresses these gaps through:

- adaptive difficulty transition (easy, medium, hard) based on answer correctness,
- per-attempt anti-cheating event capture,
- detailed result and weak-topic analytics,
- role-specific dashboarding,
- and optional live quiz mode with leaderboard updates.

3. Problem Statement

Current assessment systems used in many academic and training contexts are inefficient, static, and weak in feedback quality. Manual evaluation processes increase turnaround

time and grading inconsistency. Basic online tools often fail to provide exam-grade controls, adaptive logic, and meaningful learner analytics.

Therefore, the core problem is:

To design and implement a secure, scalable, and intelligent online quiz/examination platform that supports automated evaluation, adaptive question sequencing, anti-cheating mechanisms, and actionable performance analytics for both administrators and students.

4. Proposed Work

Quizy+ is proposed as a full-stack web solution built with Django MVT, offering complete quiz lifecycle automation and modern user experience.

4.1 Core Functional Scope

- secure registration and login,
- role-based access (Admin and Student),
- quiz creation/edit/delete,
- support for MCQ, True/False, and short-answer questions,
- timer-based attempts with auto-submit,
- objective question auto-evaluation,
- persistent result storage and retrieval.

4.2 Innovative Features

Adaptive Quiz Engine

A rule-based adaptive mechanism modifies target difficulty based on recent answer correctness:

- correct response -> harder next question,
- wrong response -> easier next question,
- nearest available difficulty chosen if exact match is unavailable.

Smart Analytics

Both student and admin dashboards provide:

- accuracy trends over attempts,
- topic-wise performance,
- weak-area identification,
- and attempt-time indicators.

Gamification

The platform includes:

- points and streak accumulation,
- badge awarding,
- and leaderboard ranking.

Anti-Cheating Controls

- tab-switch event capture,
- fullscreen-exit tracking,
- timer auto-submit,
- and controlled question navigation.

Live Quiz Mode

Admins can create join-code-based live sessions with incremental question progression and leaderboard polling.

5. System Design

5.1 Architecture Pattern

Quizy+ follows Django MVT architecture.

- Model: database entities and constraints.
- View: request handling, validation, business orchestration.
- Template: role-aware UI rendering with dynamic JavaScript behavior.

This separation promotes maintainability, testability, and incremental feature growth.

5.2 Application Modules

- config: global settings, URL routing, ASGI/WSGI configuration.
- users: custom user model, registration, login, profile.
- quiz: domain logic for quizzes, attempts, analytics, live mode, and services.
- core: landing and dashboard orchestration.

5.3 Database Schema (Conceptual)

Primary entities:

- User: extended AbstractUser with role, points, streak metadata.
- Quiz: metadata, timing, negative marking, publication controls.
- Question: type, topic, difficulty, marks, explanation.
- Option: objective options with correctness flag.
- Attempt: attempt lifecycle, timer deadline, anti-cheat counters.
- AttemptQuestion: served question order per attempt.
- AttemptAnswer: answer payload, correctness, awarded score, time spent.

- Result: summarized scoring and weak-topic outcome.
- Badge and UserBadge: gamification model.
- LiveQuizSession and LiveSessionParticipant: host-led real-time interactions.
- DailyChallenge: date-bound challenge mapping.

Design decisions:

- indexed frequently queried fields for analytics and lookup speed,
- unique constraints for active attempt consistency,
- JSON fields for flexible metadata (navigation order and weak topics).

5.4 Process Flow (Textual Diagram Description)

Student Attempt Flow

1. Student authenticates.
2. Student selects published quiz.
3. System creates or resumes active attempt.
4. Attempt serves question payload and timer state.
5. Student answers are auto-saved through asynchronous API calls.
6. Attempt is auto-submitted at expiry or manually submitted.
7. Result object is generated and analytics updated.

Admin Authoring Flow

1. Admin creates quiz metadata.
2. Admin creates questions and options.
3. Admin publishes quiz and optionally marks daily challenge.
4. Students attempt based on availability windows.
5. Admin monitors analytics and quiz health.

Adaptive Engine Flow

1. Start with medium difficulty target.
2. On correct answer, increment target level.
3. On wrong answer, decrement target level.
4. Select nearest available unserved question by target difficulty.

6. Implementation

6.1 Backend Logic (Django)

The backend uses class-based and function-based views where appropriate, with a service layer to isolate domain logic.

Implemented backend characteristics:

- role-based route protection,
- attempt ownership validation,
- structured JSON APIs for runtime quiz operations,
- robust result finalization and scoring pipeline,
- live-session control endpoints,
- PDF result export support.

6.2 Frontend Behavior (JavaScript)

The quiz player implements:

- countdown timer display and sync updates,

- question navigation grid with state highlighting,
- auto-save mechanism with payload deduplication,
- explicit save/next/submit interactions,
- anti-cheating event telemetry,
- graceful handling of expired/completed states.

UI characteristics:

- responsive card-based layout,
- consistent visual hierarchy,
- accessibility-conscious control labels,
- role-aware navigation surfaces.

6.3 Key Feature Realization

- Adaptive sequencing implemented through attempt-scoped difficulty state.
- Negative marking and objective scoring implemented per question type.
- Gamification updates linked to successful finalized results.
- Analytics generated through ORM aggregations.
- Live leaderboard served via periodic polling endpoint.

6.4 Challenges and Solutions

Challenge 1: Scoring edge cases for unanswered objective questions
Solution: objective evaluation now classifies unselected responses as unattempted instead of wrong.

Challenge 2: Duplicate active attempts in concurrent scenarios
Solution: database-level uniqueness constraints plus service-level recovery on integrity conflicts.

Challenge 3: Client-side overposting and tampering
Solution: server validates that submitted question belongs to served attempt context.

Challenge 4: Production security and test reliability conflict
Solution: hardened production settings with deterministic test-mode overrides.

7. Testing

7.1 Unit Testing

Model and service behavior verified for:

- scoring correctness,
- negative marking,
- streak updates,
- result finalization idempotency,
- adaptive difficulty transitions.

7.2 Integration Testing

End-to-end flows validated for:

- authentication pathways,
- attempt state progression,
- timer-expiry behavior,
- API payload validation,
- daily challenge and access restrictions.

7.3 Edge Case Testing

Validated edge scenarios include:

- partial submissions,
- out-of-range question navigation,
- invalid JSON payloads,
- invalid anti-cheat event injection,
- duplicate active attempt prevention,
- manual-review short-answer handling.

7.4 Security Testing

Validated controls:

- CSRF protection with tokenized POST paths,
- URL-level authorization checks,
- request tamper rejection,
- secure-cookie and HTTPS deployment settings,
- deploy-check compliance under production-like configuration.

7.5 Test Summary

- Automated tests executed: 19
- Final status: all tests passed
- Deployment security check in production-like mode: no warnings

8. Screenshots and Output

Quizy+

Home Login Get Started Theme

ADAPTIVE EXAMINATION PLATFORM

From static quizzes to intelligent assessment loops.

Quizy+ combines adaptive difficulty, anti-cheat telemetry, and deep analytics into one streamlined exam stack for institutions and creators.

Create Free Account Sign In

Adaptive	3-Level Difficulty
Security	Timer + Tab Tracking
Engagement	Badges + Streaks

Today at a Glance

Daily challenge is live

Python Starter Assessment

Baseline diagnostic quiz for Python learners.

Top Learners

student_demo	180 pts
test	45 pts

Featured Quizzes

Programming

Python Starter Assessment

Baseline diagnostic quiz for Python learners.

3 questions 15 min

Quizy+

Home Dashboard Quizzes Progress Leaderboard Profile Logout Theme

NOW ATTEMPTING

Python Starter Assessment

Time Left 14:55 Submit

Question Navigator

1 2 3

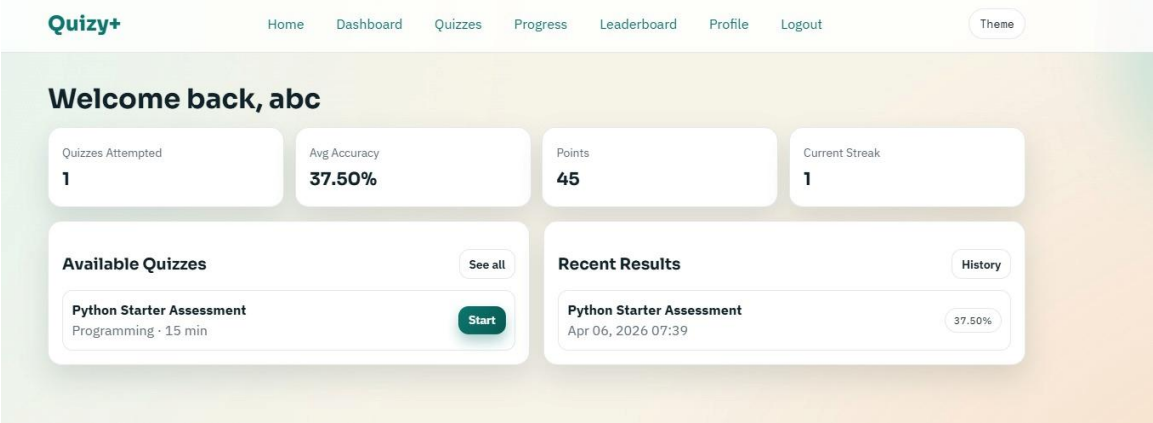
Attempted items become highlighted instantly.

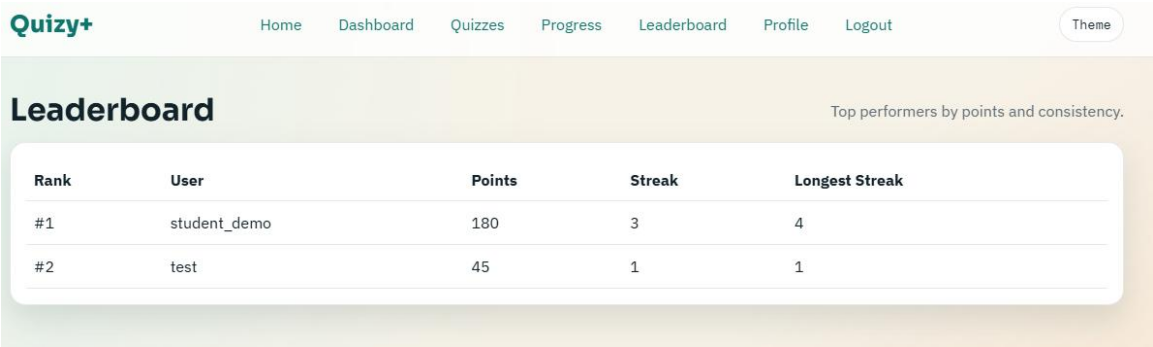
Difficulty: Medium · Topic: Data Types · Marks: 2

What built-in Python type stores key-value pairs?

Type your answer here...

Previous Save Next





9. Conclusion and Future Enhancements

Quizy+ successfully demonstrates an academically rigorous and technically robust digital examination platform with intelligent behavior, secure execution, and maintainable architecture. The project moves beyond conventional quiz systems by combining adaptive sequencing, anti-cheating telemetry, gamification, and analytics into a cohesive full-stack implementation.

Key achievements:

- complete admin-student workflow automation,
- reliable attempt lifecycle management,
- secure and validated runtime APIs,

- extensible service-based backend design,
- measurable software quality through automated testing.

Future enhancements:

- AI-assisted question generation with semantic distractor synthesis,
- NLP-based short-answer similarity scoring,
- mobile application clients for Android and iOS,
- real-time WebSocket architecture for large live sessions,
- cloud-native deployment (containerization, managed database, CDN),
- instructor-side cohort forecasting and predictive learning analytics.

References

1. Django Software Foundation. Django Documentation. https://docs.djangoproject.com/

2. Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

3. Sommerville, I. Software Engineering (10th Edition). Pearson, 2015.

4. Pressman, R. S., and Maxim, B. R. Software Engineering: A Practitioner's Approach (8th Edition). McGraw-Hill, 2014.

5. Moodle Docs. Moodle Quiz Module. https://docs.moodle.org/

6. Google Forms Help Center. Quizzes in Google Forms. https://support.google.com/docs/

7. OWASP Foundation. OWASP Top 10: The Ten Most Critical Web Application Security Risks. https://owasp.org/

8. W3C Web Accessibility Initiative. Web Content Accessibility Guidelines (WCAG). https://www.w3.org/WAI/standards-guidelines/wcag/

9. MDN Web Docs. Fetch API and Client-Side Web APIs. https://developer.mozilla.org/

10. IEEE Xplore Digital Library. Online Assessment and E-Learning Evaluation Research Articles.

Web Development Project Journal Paper: Comprehensive Research and Implementation Report

Abstract

This study presents the design, implementation, and validation of Quizy+, an intelligent online quiz and examination platform developed using Django MVT architecture with a responsive web interface. The project investigates how adaptive assessment logic, automated scoring, and role-aware analytics can improve educational evaluation quality while maintaining operational scalability and security.

Project overview

Quizy+ addresses the need for a reliable digital assessment platform that integrates exam administration, adaptive question progression, and data-driven feedback.

Key objectives

- Build a secure role-based online examination platform.
- Implement adaptive difficulty transitions.
- Automate objective scoring with negative marking support.
- Provide meaningful instructor and learner analytics.
- Validate robustness through structured testing.

Brief summary of methodologies and outcomes

A modular full-stack methodology was used: requirement analysis, architecture design, iterative implementation, and multi-layer testing. Results show successful end-to-end functionality, stable scoring behavior, strong access control, and production-ready security configuration under deployment checks.

1. Introduction

1.1 Background and Context

a. Motivation for the web development project

Digital-first learning ecosystems require examination systems that are transparent, scalable, and pedagogically informative. Existing tools often satisfy only partial requirements.

b. Problem statement

Many online quiz systems provide static questionnaires and limited analytics, reducing assessment depth and instructional value.

c. Project significance

Quizy+ contributes a practical architecture and implementation pattern for intelligent, secure, and extensible web-based assessment.

1.2 Research Objectives

a. Primary goals

- Develop a robust examination platform for real-world usage.
- Improve feedback quality through analytics and adaptive logic.

b. Specific research questions

- Can lightweight adaptive heuristics improve assessment granularity without heavy machine learning infrastructure?
- How effectively can anti-cheating telemetry and strict server validation improve attempt integrity?
- What architectural decisions maximize maintainability in a mini-project scope?

c. Scope of the project

The project targets web deployment using Django and relational storage, with optional live sessions and expandable analytics.

1.3 Thesis Statement

The central thesis is that a carefully engineered Django platform can deliver intelligent online assessment by integrating adaptive quiz logic, secure attempt lifecycle enforcement, and dual-role analytics without sacrificing maintainability or usability.

2. Literature Review

2.1 Web Development Landscape

a. Current trends in web technologies

Modern educational systems increasingly adopt API-driven backend services, responsive web UIs, and analytics-focused dashboards.

b. Emerging frameworks and tools

Django remains a strong candidate for structured server-side applications due to ORM maturity, built-in security middleware, and rapid development capability.

c. Industry best practices

Role-based access control, automated testing, secure deployment defaults, and observability-friendly architecture are now considered baseline expectations.

2.2 Related Works

a. Comparative analysis of similar projects

- Form-based systems prioritize ease but not exam integrity.

- LMS platforms provide breadth but can be operationally heavy.
- Game-focused tools prioritize engagement over deep assessment analytics.

b. Gaps in existing solutions

A gap exists for mid-complexity systems combining adaptive sequencing, anti-cheating controls, and detailed analytics in a customizable educational deployment.

3. Methodology

3.1 Research Design

a. Research approach

Applied software engineering research with iterative build-test-refine cycles.

b. Methodological framework

- Requirement elicitation
- Architecture and schema modeling
- Incremental implementation
- Multi-level testing
- Security and deployment hardening

c. Development lifecycle model

A pragmatic iterative model was adopted, enabling rapid validation of features and early defect correction.

3.2 Technology Stack

a. Frontend technologies

- HTML templates
- CSS for responsive visual system
- JavaScript for timer, auto-save, and live polling

b. Backend technologies

- Python
- Django framework with MVT architecture
- Service-layer orchestration for domain logic

c. Database and infrastructure

- SQLite (development/default)
- MySQL readiness for scaled environments
- migration-based schema evolution

d. Rationale for technology selection

The chosen stack balances rapid development, strong security defaults, clear architectural layering, and institutional deployability.

4. System Architecture

4.1 Frontend Architecture

- Component structure: template-driven views with reusable base layout.
- State management: attempt runtime state maintained in JavaScript object model.
- Routing mechanisms: server-side URL routing with API endpoints for dynamic operations.
- UI design principles: readability, clear action hierarchy, responsive behavior.

4.2 Backend Architecture

- Server-side design: role-protected views and service functions.
- API architecture: JSON endpoints for attempt question fetch, answer save, submit, and anti-cheat events.
- Database schema: normalized models for quiz domain and attempt telemetry.
- Authentication and security layers: Django auth, CSRF middleware, ownership checks, deployment hardening settings.

5. Implementation Details

5.1 Development Environment

- Python virtual environment
- Django project with modular apps
- migration-driven database lifecycle
- Git-ready repository structure

Version control strategy emphasized small, auditable, feature-focused updates with regression tests.

5.2 Core Features

- Role-aware dashboards for admin and student.
- Quiz CRUD with multi-type questions.
- Adaptive serving logic based on latest correctness.
- Timer-bound attempts with controlled finalization.
- Detailed result breakdown with weak-topic extraction.
- Gamification and leaderboard.
- Live session create/join/control flow.

5.3 Performance Optimization

- ORM-level query annotations to avoid repetitive count lookups.

- index-backed query patterns for attempts, topics, and difficulty slices.
- client autosave deduplication to reduce redundant writes.
- event throttling for anti-cheat signal endpoints.

6. User Experience and Interface Design

6.1 User Interface (UI) Design

- Design philosophy: modern, minimal, and function-forward.
- Responsive design principles: mobile-compatible grid and flexible navigation.
- Clarity measures: explicit feedback messages, state-aware buttons, and progress cues.

7. Testing and Validation

7.1 Testing Strategies

- Unit testing: model and service behavior.
- Integration testing: route and flow correctness.
- End-to-end-like validation: attempt lifecycle from start to result.

7.2 Validation Results

- Test coverage included scoring, expiry, tampering, and auth flows.
- Security checks verified production-ready settings under deployment checks.
- Issues identified during QA were fixed and locked with regression tests.

Quality assurance outcomes:

- 19 automated tests passed.
- Deployment security check passed in production-like environment.

8. Results and Discussion

8.1 Project Outcomes

a. Achievement of project objectives

All primary objectives were met:

- secure role-based workflows,
- automated and accurate evaluation,
- adaptive sequencing,
- analytics-backed feedback,
- maintainable architecture.

b. Quantitative and qualitative results

- Automated tests: 19 passing.
- Stable attempt lifecycle under edge-case scenarios.
- Reduced API overhead from autosave deduplication.

8.2 Critical Analysis

a. Strengths of the implementation

- clean modular architecture,
- strong validation and authorization patterns,
- practical intelligence features without heavy complexity.

b. Limitations and constraints

- live mode currently uses polling rather than WebSocket push,
- short-answer grading remains rule-based unless answer keys are provided,

- no built-in brute-force throttling in auth pipeline.

c. Lessons learned

- small service-layer decisions strongly influence correctness and testability,
- production security settings must be designed alongside test ergonomics,
- adaptive systems require careful edge-case handling for fairness.

9. Future Work and Recommendations

9.1 Potential Improvements

- AI-assisted question generation and distractor quality enhancement,
- semantic short-answer evaluation,
- cloud deployment with observability stack,
- horizontal scaling strategy for live sessions,
- native mobile clients for wider accessibility.

9.2 Research Implications

- Contribution to web development knowledge: demonstrates how lightweight adaptive logic can be integrated into educational platforms without ML-heavy infrastructure.
- Potential industry applications: schools, universities, certification programs, and enterprise training.

10. Conclusion

Quizy+ demonstrates that a Django-based web system can provide secure, intelligent, and scalable digital assessment when architecture, validation, and testing are treated as first-class engineering concerns. The project combines academic rigor with practical deployability and provides a strong baseline for future intelligent learning systems.

References

1. Django Documentation. https://docs.djangoproject.com/

2. OWASP Top 10. https://owasp.org/www-project-top-ten/

3. Moodle Documentation. https://docs.moodle.org/

4. Google Forms Help. https://support.google.com/docs/

5. Sommerville, I. Software Engineering. Pearson.

6. Pressman, R. S., Maxim, B. R. Software Engineering: A Practitioner's Approach.

7. MDN Web Docs. https://developer.mozilla.org/

8. W3C WCAG Standards. https://www.w3.org/WAI/standards-guidelines/wcag/

9. IEEE Xplore Research on Online Assessment Systems.

10. ACM Digital Library Articles on Adaptive Learning and Educational Technology.